



JSON

JSON (**JavaScript Object Notation**, pronounced /dʒeɪsən/ or /dʒeɪ.sɒn/) is an open standard file format and data interchange format that uses human-readable text to store and transmit data objects consisting of name–value pairs and arrays (or other serializable values). It is a commonly used data format with diverse uses in electronic data interchange, including that of web applications with servers.

JSON is a language-independent data format. It was derived from JavaScript, but many modern programming languages include code to generate and parse JSON-format data. JSON filenames use the extension `.json`.

Douglas Crockford originally specified the JSON format in the early 2000s.^[1] He and Chip Morningstar sent the first JSON message in April 2001.

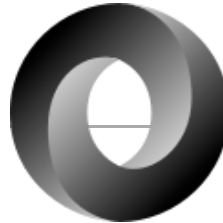
Naming and pronunciation

The 2017 international standard (ECMA-404 and ISO/IEC 21778:2017) specifies that "JSON" is "pronounced /dʒeɪ.sən/, as in 'Jason and The Argonauts'".^{[2][3]} The first (2013) edition of ECMA-404 did not address the pronunciation.^[4] Crockford said in 2011, "There's a lot of argument about how you pronounce that, but I strictly don't care."^[1] /dʒeɪ.sɒn/ is another common pronunciation.^[5]

Standards

After RFC 4627 had been available as its "informational" specification since 2006, JSON was first standardized in 2013, as ECMA-404.^[4] RFC 8259, published in 2017, is the current version of the Internet Standard STD 90, and it remains consistent with ECMA-404.^[6] That same year, JSON was also standardized as ISO/IEC 21778:2017.^[2] The ECMA and ISO/IEC standards describe only the allowed syntax, whereas the RFC covers some security and interoperability considerations.^[7]

JavaScript Object Notation



The JSON logo is a Möbius strip

<u>Filename extension</u>	<code>.json</code>
<u>Internet media type</u>	<code>application/json</code>
<u>Type code</u>	TEXT
<u>Uniform Type Identifier (UTI)</u>	<code>public.json</code>
<u>Type of format</u>	Data interchange
<u>Extended from</u>	<u>JavaScript</u>
<u>Standard</u>	STD 90 (https://tools.ietf.org/html/std90) (RFC 8259 (https://www.rfc-editor.org/rfc/rfc8259)), <u>ECMA-404</u> (https://ecma-international.org/publications-and-standards/standards/ecma-404/), <u>ISO/IEC 21778:2017</u> (https://www.iso.org/standard/71616.html)
<u>Open format?</u>	Yes
<u>Website</u>	json.org (https://json.org/)

History

JSON grew out of a need for a real-time server-to-browser session communication protocol without using browser plugins such as Flash or Java applets, the dominant methods used in the early 2000s.^[8]

Crockford first specified and popularized the JSON format.^[1] The acronym originated at State Software, a company cofounded by Crockford and others in March 2001. The cofounders agreed to build a system that used standard browser capabilities and provided an abstraction layer for Web developers to create stateful Web applications that had a persistent duplex connection to a Web server by holding two Hypertext Transfer Protocol (HTTP) connections open and recycling them before standard browser time-outs if no further data were exchanged. The cofounders had a round-table discussion and voted on whether to call the data format JSML (JavaScript Markup Language) or JSON (JavaScript Object Notation), as well as under what license type to make it available. The JSON.org^[9] website was launched in 2001. In December 2005, Yahoo! began offering some of its Web services in JSON.^[10]



Douglas Crockford at the Yahoo Building (2007)

A precursor to the JSON libraries was used in a children's digital asset trading game project named Cartoon Orbit at Communities.com which used a browser side plug-in with a proprietary messaging format to manipulate DHTML elements. Upon discovery of early Ajax capabilities, digiGroups, Noosh, and others used frames to pass information into the user browsers' visual field without refreshing a Web application's visual context, realizing real-time rich Web applications using only the standard HTTP, HTML, and JavaScript capabilities of Netscape 4.0.5+ and Internet Explorer 5+. Crockford then found that JavaScript could be used as an object-based messaging format for such a system. The system was sold to Sun Microsystems, Amazon.com, and EDS.

JSON was based on a subset of the JavaScript scripting language (specifically, Standard ECMA-262 3rd Edition—December 1999^[11]) and is commonly used with JavaScript, but it is a language-independent data format. Code for parsing and generating JSON data is readily available in many programming languages. JSON's website lists JSON libraries by language.

In October 2013, Ecma International published the first edition of its JSON standard ECMA-404.^[4] That same year, RFC 7158 (<https://www.rfc-editor.org/rfc/rfc7158>) used ECMA-404 as a reference. In 2014, RFC 7159 (<https://www.rfc-editor.org/rfc/rfc7159>) became the main reference for JSON's Internet uses, superseding RFC 4627 (<https://www.rfc-editor.org/rfc/rfc4627>) and RFC 7158 (<https://www.rfc-editor.org/rfc/rfc7158>) (but preserving ECMA-262 and ECMA-404 as main references). In November 2017, ISO/IEC JTC 1/SC 22 published ISO/IEC 21778:2017^[2] as an international standard. On December 13,

2017, the [Internet Engineering Task Force](https://www.rfc-editor.org/rfc/rfc7159) [obsoleted RFC 7159](https://www.rfc-editor.org/rfc/rfc7159) (<https://www.rfc-editor.org/rfc/rfc7159>) when it published [RFC 8259](https://www.rfc-editor.org/rfc/rfc8259) (<https://www.rfc-editor.org/rfc/rfc8259>), which is the current version of the Internet Standard STD 90.^{[12][13]}

Crockford added a clause to the JSON license stating, "The Software shall be used for Good, not Evil", in order to [open-source](#) the JSON libraries while mocking corporate lawyers and those who are overly pedantic. On the other hand, this clause led to [license compatibility](#) problems of the JSON license with other [open-source licenses](#) since [open-source software](#) and [free software](#) usually imply no restrictions on the purpose of use.^[14]

Syntax

The following example shows a possible JSON representation describing a person.

```
{
  "first_name": "John",
  "last_name": "Smith",
  "is_alive": true,
  "age": 27,
  "address": {
    "street_address": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postal_code": "10021-3100"
  },
  "phone_numbers": [
    {
      "type": "home",
      "number": "212 555-1234"
    },
    {
      "type": "office",
      "number": "646 555-4567"
    }
  ],
  "children": [
    "Catherine",
    "Thomas",
    "Trevor"
  ],
  "spouse": null
}
```

Character encoding

Although Crockford originally asserted that JSON is a strict subset of [JavaScript](#) and [ECMAScript](#),^[15] his specification actually allows valid JSON documents that are not valid JavaScript; JSON allows the [Unicode line terminators](#) [U+2028](#) LINE SEPARATOR and [U+2029](#) PARAGRAPH SEPARATOR to appear unescaped in quoted strings, while ECMAScript 2018 and older do not.^{[16][17]} This is a consequence of JSON disallowing only "control characters". For maximum portability, these characters are backslash-escaped.

JSON exchange in an open ecosystem must be encoded in [UTF-8](#).^[6] The encoding supports the full Unicode character set, including those characters outside the [Basic Multilingual Plane](#) (U+0000 to U+FFFF). However, if escaped, those characters must be written using [UTF-16 surrogate pairs](#). For example, to include the [Emoji](#) character [U+1F610](#) 😊 NEUTRAL FACE in JSON:

```
{ "face": "😞" }
```

Or:

```
{ "face": "\uD83D\uDE10" }
```

JSON became a strict subset of ECMAScript as of the language's 2019 revision.^{[17][18]}

Data types

JSON's basic data types are:

- **Number**: a signed decimal number that may contain a fractional part and may use exponential E notation but cannot include non-numbers such as NaN. The format makes no distinction between integer and floating-point. JavaScript uses IEEE-754 double-precision floating-point format for all its numeric values (later also supporting BigInt^[19]), but other languages implementing JSON may encode numbers differently.
- **String**: a sequence of zero or more Unicode characters. Strings are delimited with double quotation marks and support a backslash escaping syntax.
- **Boolean**: either of the values `true` or `false`
- **Array**: an ordered list of zero or more elements, each of which may be of any type. Arrays use square bracket notation with comma-separated elements.
- **Object**: a collection of name–value pairs where the names (also called keys) are strings. The current ECMA standard states, "The JSON syntax does not impose any restrictions on the strings used as names, does not require that name strings be unique, and does not assign any significance to the ordering of name/value pairs."^[20] Objects are delimited with curly brackets and use commas to separate each pair, while within each pair, the colon ":" character separates the key or name from its value.
- **null**: an empty value, using the word `null`

Whitespace is allowed and ignored around or between syntactic elements (values and punctuation, but not within a string value). Four specific characters are considered whitespace for this purpose: space, horizontal tab, line feed, and carriage return. In particular, the byte order mark must not be generated by a conforming implementation (though it may be accepted when parsing JSON). JSON does not provide syntax for comments.^[21]

Early versions of JSON (such as specified by RFC 4627 (<https://www.rfc-editor.org/rfc/rfc4627>)) required that a valid JSON text must consist of only an object or an array type, which could contain other types within them. This restriction was dropped in RFC 7158 (<https://www.rfc-editor.org/rfc/rfc7158>), where a JSON text was redefined as any serialized value.

Numbers in JSON are agnostic with regard to their representation within programming languages. While this allows for numbers of arbitrary precision to be serialized, it may lead to portability issues. For example, since no differentiation is made between integer and floating-point values, some implementations may treat `42`, `42.0`, and `4.2E+1` as the same number, while others may not. The JSON standard makes no requirements regarding implementation details such as overflow, underflow, loss of precision, rounding, or signed zeros, but it does recommend expecting no more than IEEE 754 binary64 precision for "good interoperability". There is no inherent precision loss in serializing a

machine-level binary representation of a floating-point number (like binary64) into a human-readable decimal representation (like numbers in JSON) and back; there exist published algorithms to do this conversion exactly and optimally.^[22]

Comments were intentionally excluded from JSON. In 2012, Douglas Crockford described his design decision thus: "I removed comments from JSON because I saw people were using them to hold parsing directives, a practice which would have destroyed interoperability."^[21]

JSON disallows "trailing commas", a comma after the last value inside a data structure.^[23] Trailing commas are a common feature of JSON derivatives to improve ease of use.^[24]

Interoperability

RFC 8259 (<https://www.rfc-editor.org/rfc/rfc8259>) describes certain aspects of JSON syntax that, while legal per the specifications, can cause interoperability problems.

- Certain JSON implementations only accept JSON texts representing an object or an array. For interoperability, applications interchanging JSON should transmit messages that are objects or arrays.
- The specifications allow JSON objects that contain multiple members with the same name. The behavior of implementations processing objects with duplicate names is unpredictable. For interoperability, applications should avoid duplicate names when transmitting JSON objects.
- The specifications specifically say that the order of members in JSON objects is not significant. For interoperability, applications should avoid assigning meaning to member ordering even if the parsing software makes that ordering visible.
- While the specifications place no limits on the magnitude or precision of JSON number literals, the widely used JavaScript implementation stores them as IEEE754 "binary64" quantities. For interoperability, applications should avoid transmitting numbers that cannot be represented in this way, for example, 1E400 or 3.141592653589793238462643383279.
- While the specifications do not constrain the character encoding of the Unicode characters in a JSON text, the vast majority of implementations assume UTF-8 encoding; for interoperability, applications should always and only encode JSON messages in UTF-8.
- The specifications do not forbid transmitting byte sequences that incorrectly represent Unicode characters. For interoperability, applications should transmit messages containing no such byte sequences.
- The specification does not constrain how applications go about comparing Unicode strings. For interoperability, applications should always perform such comparisons code unit by code unit.

In 2015, the IETF published RFC 7493 (<https://www.rfc-editor.org/rfc/rfc7493>), describing the "I-JSON Message Format", a restricted profile of JSON that constrains the syntax and processing of JSON to avoid, as much as possible, these interoperability issues.

Semantics

While JSON provides a syntactic framework for data interchange, unambiguous data interchange also requires agreement between producer and consumer on the semantics of specific use of the JSON syntax.^[25] One example of where such an agreement is necessary is the serialization of data types that are

not part of the JSON standard, for example, dates and regular expressions.

Metadata and schema

The official MIME type for JSON text is `application/json`,^[26] and most modern implementations have adopted this. Legacy MIME types include `text/json`, `text/x-json`, and `text/javascript`.^[27] The standard filename extension is `.json`.^[28]

JSON Schema specifies a JSON-based format to define the structure of JSON data for validation, documentation, and interaction control. It provides a contract for the JSON data required by a given application and how that data can be modified.^[29] JSON Schema is based on the concepts from XML Schema (XSD) but is JSON-based. As in XSD, the same serialization/deserialization tools can be used both for the schema and data, and it is self-describing. It is specified in an Internet Draft at the IETF, with the latest version as of 2024 being "Draft 2020-12".^[30] There are several validators available for different programming languages,^[31] each with varying levels of conformance.

The JSON standard does not support object references, but an IETF draft standard for JSON-based object references exists.^[32]

Uses

JSON-RPC is a remote procedure call (RPC) protocol built on JSON, as a replacement for XML-RPC or SOAP. It is a simple protocol that defines only a handful of data types and commands. JSON-RPC lets a system send notifications (information to the server that does not require a response) and multiple calls to the server that can be answered out of order.

Asynchronous JavaScript and JSON (or AJAX) refers to the same dynamic web page methodology as Ajax, but instead of XML, JSON is the data format. AJAX is a web development technique that provides for the ability of a web page to request new data after it has loaded into the web browser. Typically, it renders new data from the server in response to user actions on that web page. For example, what the user types into a search box, client-side code then sends to the server, which immediately responds with a drop-down list of matching database items.

JSON has seen ad hoc usage as a configuration language. However, it does not support comments. In 2012, Douglas Crockford, JSON creator, had this to say about comments in JSON when used as a configuration language: "I know that the lack of comments makes some people sad, but it shouldn't. Suppose you are using JSON to keep configuration files, which you would like to annotate. Go ahead and insert all the comments you like. Then pipe it through JSMIn^[33] before handing it to your JSON parser."^[21]

MongoDB uses JSON-like data for its document-oriented database.

Some relational databases have added support for native JSON data types, such as JSONB in PostgreSQL^[34] and JSON in MySQL.^[35] This allows developers to insert JSON data directly without having to convert it to another format.

Safety

JSON being a subset of JavaScript can lead to the misconception that it is safe to pass JSON texts to the JavaScript `eval()` function. This is not safe, due to certain valid JSON texts, specifically those containing `U+2028` LINE SEPARATOR or `U+2029` PARAGRAPH SEPARATOR, not being valid JavaScript code until JavaScript specifications were updated in 2019, and so older engines may not support it.^[36] To avoid the many pitfalls caused by executing arbitrary code from the Internet, a new function, `JSON.parse()`, was first added to the fifth edition of ECMAScript,^[37] which as of 2017 is supported by all major browsers. For non-supported browsers, an API-compatible JavaScript library is provided by Douglas Crockford.^[38] In addition, the TC39 proposal "Subsume JSON" made ECMAScript a strict JSON superset as of the language's 2019 revision.^{[17][18]} Various JSON parser implementations have suffered from denial-of-service attack and mass assignment vulnerability.^{[39][40]}

Alternatives

JSON is promoted as a low-overhead alternative to XML as both of these formats have widespread support for creation, reading, and decoding in the real-world situations where they are commonly used.^[41] Apart from XML, examples could include CSV and supersets of JSON. Google Protocol Buffers can fill this role, although it is not a data interchange language. CBOR has a superset of the JSON data types, but it is not text-based. Ion is also a superset of JSON, with a wider range of primary types, annotations, comments, and allowing trailing commas.^[42]

XML

XML has been used to describe structured data and to serialize objects. Various XML-based protocols exist to represent the same kind of data structures as JSON for the same kind of data interchange purposes. Data can be encoded in XML in several ways. The most expansive form using tag pairs results in a much larger (in character count) representation than JSON, but if data is stored in attributes and *short tag* form where the closing tag is replaced with `/>`, the representation is often about the same size as JSON or just a little larger. However, an XML attribute can only have a single value and each attribute can appear at most once on each element.

XML separates data from metadata (via the use of elements and attributes), while JSON does not have such a concept.

Another key difference is the addressing of values. JSON has objects with a simple key-to-value mapping, whereas in XML addressing happens on *nodes*, each of which receives a unique ID via the XML processor. Additionally, the XML standard defines a common attribute `xml:id`, that can be used by the user, to set an ID explicitly.

XML tag names cannot contain any of the characters `!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~`, nor a space character, and cannot begin with `-`, `.`, or a numeric digit, whereas JSON keys can (even if quotation mark and backslash must be escaped).^[43]

XML values are strings of *characters*, with no built-in type safety. XML has the concept of schema, that permits strong typing, user-defined types, predefined tags, and formal structure, allowing for formal validation of an XML stream. JSON has several types built-in and has a similar schema concept in JSON Schema.

XML supports comments, while JSON does not.^{[44][21]}

Supersets

Support for comments and other features have been deemed useful, which has led to several nonstandard JSON supersets being created. Among them are HJSON,^[45] HOCON, and JSON5 (which despite its name, is not the fifth version of JSON).^{[46][47]}

YAML

YAML version 1.2 is a superset of JSON; prior versions were not strictly compatible. For example, escaping a slash / with a backslash \ is valid in JSON, but was not valid in YAML.^[48] YAML supports comments, while JSON does not.^{[48][46][21]}

CSON

CSON ("CoffeeScript Object Notation") uses significant indentation and unquoted keys, and assumes an outer object declaration. It was used for configuring GitHub's Atom text editor.^{[49][50][51]}

There is also an unrelated project called **CSON** ("Cursive Script Object Notation") that is more syntactically similar to JSON.^[52]

HOCON

HOCON ("Human-Optimized Config Object Notation") is a format for human-readable data, and a superset of JSON.^[53] The uses of HOCON are:

- It is used mostly along with the Play Framework,^[54] and is developed by Lightbend.
- It is also supported as a configuration format for .NET projects via Akka.NET^{[55][56]} and Puppet.^[57]
- TIBCO Streaming:^[58] HOCON is the primary configuration file format for the TIBCO Streaming^[59] family of products (StreamBase, LiveView, and Artifact Management Server) as of TIBCO Streaming Release 10.^[60]
- It is also the primary configuration file format for several subsystems of Exabeam Advanced Analytics.^[61]
- Jitsi uses it as the "new" config system and .properties-Files as fallback^{[62][63]}

JSON5

JSON5 ("JSON5 Data Interchange Format") is an extension of JSON syntax that, like JSON, is also valid JavaScript syntax. The specification was started in 2012 and finished in 2018 with version 1.0.0.^[64] The main differences to JSON syntax are:

- Optional trailing commas
- Unquoted object keys
- Single quoted and multiline strings
- Additional number formats
- Comments

JSON5 syntax is supported in some software as an extension of JSON syntax, for instance in [SQLite](#).^[65]

JSONC

JSONC (JSON with Comments) is a subset of JSON5 used in Microsoft's [Visual Studio Code](#).^[66]

- supports single-line comments (`//`) and block comments (`/* */`)
- accepts trailing commas, but they are discouraged and the editor will display a warning

Derivatives

Several serialization formats have been built on or from the JSON specification. Examples include

- [GeoJSON](#), a format designed for representing simple geographical features^{[67][68]}
- [JSON-LD](#), a method of encoding [linked data](#) using JSON^{[69][70]}
- [JSON-RPC](#), a remote procedure call protocol encoded in JSON^[71]
- [JsonML](#), a lightweight markup language used to map between [XML](#) and JSON^{[72][73]}
- [Smile \(data interchange format\)](#)^{[74][75]}
- [UBJSON](#), a binary computer data interchange format imitating JSON, but requiring fewer bytes of data^{[76][77]}
- [Jsonnet](#) is a [prototype-based](#) domain-specific language that produces JSON files. All JSON documents are valid Jsonnet programs that will be emitted unchanged when run. Jsonnet extends JSON by supporting variables, imports, loops, comments, etc.^{[78][79]} Jsonnet is used as a configuration language for cloud infrastructure engineering.^[80]

See also

- [BSON](#)
- [Comparison of data serialization formats](#)
- [Extensible Data Notation](#)
- [Amazon Ion](#) – a superset of JSON (though limited to [UTF-8](#), like JSON for interchange, unlike general JSON)
- [Jackson \(API\)](#)
- [jaql](#) – a functional data processing and query language most commonly used for JSON

query processing

- jq – a "JSON query language" and high-level programming language
- JSONiq – a JSON-oriented query and processing language based on XQuery
- JSON streaming
- S-expression

References

1. Crockford, Douglas (August 28, 2011). *The JSON Saga* (<https://www.youtube.com/watch?v=-C-JoyNuQJs>) (video). YUI Library. Retrieved February 21, 2022 – via YouTube.
 - Transcript (<https://web.archive.org/web/20191030002009/http://transcriptvids.com/v/-C-JoyNuQJs.html>) (archived 2019-10-30) from Transcript Vids
2. "ISO/IEC 21778:2017 Information technology — The JSON data interchange syntax" (<http://www.iso.org/standard/71616.html>). ISO. Retrieved June 24, 2025.
3. "ECMA-404: The JSON Data Interchange Syntax" (<https://ecma-international.org/publications-and-standards/standards/ecma-404/>) (2nd ed.). Ecma International. December 2017. p. iii, footnote. Archived (<https://web.archive.org/web/20191027160438/http://www.ecma-international.org/publications/files/ECMA-ST/ECMA-404.pdf>) (PDF) from the original on October 27, 2019. Retrieved April 29, 2024.
4. "ECMA-404: The JSON Data Interchange Format" (<https://ecma-international.org/publications-and-standards/standards/ecma-404/>) (1st ed.). Ecma International. October 2013. Archived (<https://web.archive.org/web/20131101200049/http://www.ecma-international.org/publications/files/ECMA-ST/ECMA-404.pdf>) (PDF) from the original on November 1, 2013. Retrieved November 20, 2023.
5. Sweigart, Al (2015). *Automate the Boring Stuff with Python: Practical Programming for Total Beginners* (<https://books.google.com/books?id=8AcvDwAAQBAJ&pg=PA319>). No Starch Press. p. 319. ISBN 9781593276850. Retrieved October 29, 2019. "pronounced 'JAY-sawn' or 'Jason'—it doesn't matter how because either way people will say you're pronouncing it wrong" (assuming the cot-caught merger: "sawn" /sɔːn/ → [sɒn]).
6. Bray, T. (December 2017). Bray, T (ed.). *The JavaScript Object Notation (JSON) Data Interchange Format* (<https://tools.ietf.org/html/rfc8259>). IETF. doi:10.17487/RFC8259 (<http://doi.org/10.17487%2FRFC8259>). S2CID 263868313 (<https://api.semanticscholar.org/CorpusID:263868313>). RFC 8259 (<https://datatracker.ietf.org/doc/html/rfc8259>). Retrieved February 16, 2018.
7. Bray, Tim. "JSON Redux AKA RFC7159" (<https://www.tbray.org/ongoing/When/201x/2014/03/05/RFC7159-JSON>). *Ongoing*. Retrieved March 16, 2014.
8. "Unofficial Java History" (<https://web.archive.org/web/20140526235903/http://www.edu4java.com/en/java/unofficial-java-history.html>). Edu4Java. 26 May 2014. Archived from the original (<http://www.edu4java.com/en/java/unofficial-java-history.html>) on 26 May 2014. Retrieved 30 August 2019. "In 1996, Macromedia launches Flash technology which occupies the space left by Java and ActiveX, becoming the de facto standard for animation on the client side."
9. "JSON" (<http://json.org/>). *json.org*.
10. Yahoo!. "Using JSON with Yahoo! Web services" (<https://web.archive.org/web/20071011085815/http://developer.yahoo.com/common/json.html>). Archived from the original (<http://developer.yahoo.com/common/json.html>) on October 11, 2007. Retrieved July 3, 2009.
11. Crockford, Douglas (May 28, 2009). "Introducing JSON" (<http://json.org/>). json.org. Retrieved July 3, 2009. "It is based on a subset of the JavaScript Programming Language, Standard ECMA-262 3rd Edition - December 1999."

12. Bray, Tim (December 2017). "History for draft-ietf-jsonbis-rfc7159bis-04" (<https://datatracker.ietf.org/doc/rfc8259/history/>). *IETF Datatracker*. Internet Engineering Task Force. Retrieved October 24, 2019. "2017-12-13 [...] RFC published"
13. Bray, Tim (December 13, 2017). "RFC 8259 - The JavaScript Object Notation (JSON) Data Interchange Format" (<https://datatracker.ietf.org/doc/rfc8259/>). *IETF Datatracker*. Internet Engineering Task Force. Retrieved October 24, 2019. "Type: RFC - Internet Standard (December 2017; Errata); Obsoletes RFC 7159; Also known as STD 90"
14. "Apache and the JSON license (<https://lwn.net/Articles/707510/>)" on LWN.net by Jake Edge (November 30, 2016).
15. Douglas Crockford (July 10, 2016). "JSON in JavaScript" (<https://web.archive.org/web/20160710230817/http://www.json.org/js.html>). Archived from the original (<http://www.json.org/js.html>) on July 10, 2016. Retrieved August 13, 2016. "JSON is a subset of the object literal notation of JavaScript."
16. Holm, Magnus (May 15, 2011). "JSON: The JavaScript subset that isn't" (<https://web.archive.org/web/20120513012409/http://timelessrepo.com/json-isnt-a-javascript-subset>). The timeless repository. Archived from the original (<http://timelessrepo.com/json-isnt-a-javascript-subset>) on May 13, 2012. Retrieved September 23, 2016.
17. "Subsume JSON: Proposal to make all JSON text valid ECMA-262" (<https://tc39.es/proposal-json-superset/>). Ecma TC39. August 23, 2019. Retrieved August 27, 2019.
18. "Advance to Stage 4 - tc39/proposal-json-superset" (<https://github.com/tc39/proposal-json-superset/commit/0604b6083e18fe033a1520388b8c6146bcd79e23>). *GitHub*. May 22, 2018.
19. "BigInt - MDN Web doc glossary" (<https://developer.mozilla.org/en-US/docs/Glossary/BigInt>). *Mozilla*. Retrieved October 18, 2020.
20. ECMA-404, 2nd ed., p. 3: "The JSON syntax does not impose any restrictions on the strings used as names, does not require that name strings be unique, and does not assign any significance to the ordering of name/value pairs."
21. Crockford, Douglas (April 30, 2012). "Comments in JSON" (<https://archive.today/20150704102718/https://plus.google.com/+DouglasCrockfordEsq/posts/RK8qyGVaGSr>). Archived from the original (<https://plus.google.com/+DouglasCrockfordEsq/posts/RK8qyGVaGSr>) on July 4, 2015. Retrieved August 30, 2019. "I removed comments from JSON because I saw people were using them to hold parsing directives, a practice which would have destroyed interoperability. I know that the lack of comments makes some people sad, but it shouldn't. Suppose you are using JSON to keep configuration files, which you would like to annotate. Go ahead and insert all the comments you like. Then pipe it through JSMin before handing it to your JSON parser."
22. Andryscio, Marc; Jhala, Ranjit; Lerner, Sorin (February 18, 2016). Printing Floating-Point Numbers: An Always Correct Method (<https://cseweb.ucsd.edu/~mandrysc/pub/dtoa.pdf>) (PDF) (Report). Retrieved May 31, 2025.
23. "Trailing commas - JavaScript | MDN" (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Trailing_commas). *developer.mozilla.org*. September 12, 2023. Retrieved December 16, 2023.
24. "JSON5" (<https://web.archive.org/web/20201129113821/https://json5.org/>). json5. Archived from the original (<https://json5.org/>) on November 29, 2020. Retrieved December 16, 2020.
25. ECMA-404, 2nd ed., p. iii: "The JSON syntax is not a specification of a complete data interchange. Meaningful data interchange requires agreement between a producer and consumer on the semantics attached to a particular use of the JSON syntax. What JSON does provide is the syntactic framework to which such semantics can be attached"
26. "Media Types" (<https://www.iana.org/assignments/media-types/application/index.html>). *iana.org*. Retrieved September 13, 2015.
27. "Correct Content-Type Header for JSON" (<https://reqbin.com/req/abghm4zf/json-content-type>). *ReqBin*. January 13, 2023. Retrieved March 23, 2024.

28. Bray, Tim (December 2017). Bray, T. (ed.). "11. IANA Considerations" (<https://datatracker.ietf.org/doc/html/rfc8259#section-11>). *RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format*. IETF. doi:10.17487/RFC8259 (<https://doi.org/10.17487%2FRFC8259>). S2CID 263868313 (<https://api.semanticscholar.org/CorpusID:263868313>).
29. "JSON Schema and Hyper-Schema" (<https://json-schema.org>). *json-schema.org*. Retrieved June 8, 2021.
30. "JSON Schema - Specification Links" (<https://json-schema.org/specification-links>). *json-schema.org*. Retrieved March 22, 2024.
31. "JSON Schema Implementations" (<https://web.archive.org/web/20210616234051/http://json-schema.org/implementations.html>). *json-schema.org*. Archived from the original (<https://json-schema.org/implementations.html>) on June 16, 2021. Retrieved June 8, 2021.
32. Zyp, Kris (September 16, 2012). Bryan, Paul C. (ed.). "JSON Reference: draft-pbryan-zyp-json-ref-03" (<https://datatracker.ietf.org/doc/html/draft-pbryan-zyp-json-ref-03>). *Internet Engineering Task Force*.
33. Crockford, Douglas (May 16, 2019). "JSMIn" (<https://www.crockford.com/jsmin.html>). Retrieved August 12, 2020. "JSMIn [2001] is a minification tool that removes comments and unnecessary whitespace from JavaScript files."
34. "JSONB data type" ([https://www.cockroachlabs.com/docs/stable/jsonb#:~:text=The%20JSONB%20data%20type%20stores%20JSON%20\(JavaScript,JSONB%20value%2C%20which%20eliminates%20whitespace%2C%20duplicate%20keys%2C](https://www.cockroachlabs.com/docs/stable/jsonb#:~:text=The%20JSONB%20data%20type%20stores%20JSON%20(JavaScript,JSONB%20value%2C%20which%20eliminates%20whitespace%2C%20duplicate%20keys%2C)). *www.cockroachlabs.com*. Retrieved April 1, 2025.
35. "The JSON data type" (<https://dev.mysql.com/doc/refman/8.4/en/json.html>). *dev.mysql.com*. Retrieved April 1, 2025.
36. "JSON: The JavaScript subset that isn't" (<https://web.archive.org/web/20120513012409/http://timelessrepo.com/json-isnt-a-javascript-subset>). Magnus Holm. Archived from the original (<http://timelessrepo.com/json-isnt-a-javascript-subset>) on May 13, 2012. Retrieved May 16, 2011.
37. "ECMA-262: ECMAScript Language Specification" (<https://ecma-international.org/publications-and-standards/standards/ecma-262/>) (5th ed.). December 2009. Archived (<https://web.archive.org/web/20110414214458/http://www.ecma-international.org/publications/files/ECMA-ST/ECMA-262.pdf>) (PDF) from the original on April 14, 2011. Retrieved March 18, 2011.
38. "douglascrockford/JSON-js" (<https://github.com/douglascrockford/JSON-js/blob/master/json2.js>). *GitHub*. August 13, 2019.
39. "Denial of Service and Unsafe Object Creation Vulnerability in JSON (CVE-2013-0269)" (<https://www.ruby-lang.org/en/news/2013/02/22/json-dos-cve-2013-0269/>). Retrieved January 5, 2016.
40. "Microsoft .NET Framework JSON Content Processing Denial of Service Vulnerability" (<https://web.archive.org/web/20181106233952/http://tools.cisco.com/security/center/viewAlert.x?alertId=31048>). Archived from the original (<http://tools.cisco.com/security/center/viewAlert.x?alertId=31048>) on November 6, 2018. Retrieved January 5, 2016.
41. "JSON: The Fat-Free Alternative to XML" (<http://www.json.org/xml.html>). *json.org*. Retrieved March 14, 2011.
42. "Amazon Ion" (<https://amazon-ion.github.io/ion-docs/index.html>). Amazon. Archived (<https://web.archive.org/web/20240812225059/http://amazon-ion.github.io/ion-docs/index.html>) from the original on August 12, 2024. Retrieved August 26, 2024.
43. "XML 1.1 Specification" (<https://www.w3.org/TR/xml11/>). World Wide Web Consortium. Retrieved August 26, 2019.
44. Saternos, Casimir (2014). *Client-server web apps with Javascript and Java*. "O'Reilly Media, Inc.". p. 45. ISBN 9781449369316.

45. Edelman, Jason; Lowe, Scott; Oswalt, Matt. *Network Programmability and Automation*. O'Reilly Media. "for data representation you can pick one of the following: YAML, YAMLEX, JSON, JSON5, HJSON, or even pure Python"
46. McCombs, Thayne (July 16, 2018). "Why JSON isn't a good configuration language" (<http://www.lucidchart.com/techblog/2018/07/16/why-json-isnt-a-good-configuration-language/>). Lucid Chart. Retrieved June 15, 2019.
47. "HOCON (Human-Optimized Config Object Notation)" (<https://github.com/lightbend/config/blob/master/HOCON.md>). *GitHub*. January 28, 2019. Retrieved August 28, 2019. "The primary goal is: keep the semantics (tree structure; set of types; encoding/escaping) from JSON, but make it more convenient as a human-editable config file format."
48. "YAML Ain't Markup Language (YAML) Version 1.2" (<http://www.yaml.org/spec/1.2/spec.html>). *yaml.org*. Retrieved September 13, 2015.
49. Dohm, Lee (2014). "CoffeeScript Object Notation" (<https://www.lee-dohm.com/big-book-of-atom/1-introduction/30-coffeescript-object-notation.html>). *The Big Book of Atom*. Archived (<https://web.archive.org/web/20230422101822/http://www.lee-dohm.com/big-book-of-atom/1-introduction/30-coffeescript-object-notation.html>) from the original on April 22, 2023. Retrieved April 29, 2024.
50. "Basic Customization" (<https://flight-manual.atom-editor.cc/using-atom/sections/basic-customization/#configuring-with-cson>). *Atom Flight Manual*. *GitHub*. Archived (<https://web.archive.org/web/20240429181349/https://flight-manual.atom-editor.cc/using-atom/sections/basic-customization/>) from the original on April 29, 2024. Retrieved April 29, 2024.
51. "CSON" (<https://github.com/bevry/cson>). Bevry. December 20, 2023. Archived (<https://web.archive.org/web/20240423234311/http://github.com/bevry/cson>) from the original on April 23, 2024. Retrieved April 29, 2024 – via *GitHub*.
52. Seonghoon, Kang (July 1, 2021). "CSON" (<https://github.com/lifthrasiir/cson>). Archived (<https://web.archive.org/web/20231216044815/http://github.com/lifthrasiir/cson>) from the original on December 16, 2023. Retrieved February 27, 2023 – via *GitHub*.
53. "config/HOCON.md at master · lightbend/config" (<https://github.com/lightbend/config>). *GitHub*. Retrieved August 5, 2021.
54. "Config File - 2.5.x" (<https://www.playframework.com/documentation/2.5.x/ConfigFile>). *www.playframework.com*. Retrieved August 5, 2021.
55. Akka.NET HOCON Docs (<https://getakka.net/articles/hocon/index.html>)
56. "Akka.NET Documentation" (<https://getakka.net/>). *getakka.net*. Retrieved August 5, 2021.
57. "Managing HOCON configuration files with Puppet" (<https://web.archive.org/web/20170211075556/https://puppet.com/blog/managing-hocon-configuration-files-puppet>). Archived from the original (<https://puppet.com/blog/managing-hocon-configuration-files-puppet>) on February 11, 2017. Retrieved March 4, 2023.
58. "StreamBase Documentation" (<https://docs.streambase.com/latest/index.jsp>). *docs.streambase.com*. Retrieved August 5, 2021.
59. "Configuration Guide" (<https://docs.streambase.com/latest/topic/com.streambase.sb.ide.help/data/html/hocon/index.html>). *docs.streambase.com*. Retrieved August 5, 2021.
60. "StreamBase New and Noteworthy Archive" (<https://docs.streambase.com/latest/topic/com.streambase.sb.ide.help/data/html/dochome/xarchive-sb-noteworthy.html>). *docs.streambase.com*. Retrieved August 5, 2021.
61. "Exabeam Advanced Analytics Release Notes" (<https://web.archive.org/web/20201020195403/https://docs.exabeam.com/en/advanced-analytics/i48/release-notes/109536-what-s-new.html>). Archived from the original (<https://docs.exabeam.com/en/advanced-analytics/i48/release-notes/109536-what-s-new.html>) on October 20, 2020. Retrieved March 4, 2023.
62. JITSI Project. "Config phase 1" (<https://github.com/jitsi/jitsi-videobridge/commit/676fb3df696c4737ac086fb856af1f85c6ded90e>). *GitHub*. Retrieved February 16, 2021.

63. JITSI Project. "reference.conf" (<https://github.com/jitsi/jicofo/blob/add-selection-test/src/main/resources/reference.conf>). *GitHub*. Retrieved February 16, 2021.
64. "The JSON5 Data Interchange Format" (<https://spec.json5.org/>). Retrieved June 25, 2022.
65. SQLite. "JSON Functions And Operators" (https://www.sqlite.org/json1.html#json5_extensions). Retrieved June 25, 2023.
66. "JSON with Comments - JSON editing in Visual Studio Code" (https://code.visualstudio.com/docs/languages/json#_json-with-comments). *Visual Studio Code*. Microsoft. Retrieved April 29, 2024.
67. Butler, H.; Daly, M.; Doyle, A.; Gillies, Sean; Schaub, T.; Hagen, Stefan (August 2016). "RFC 7946 - The GeoJSON Format" (<https://datatracker.ietf.org/doc/html/rfc7946>). *IETF Datatracker*. Retrieved June 17, 2022.
68. "GeoJSON" (<https://geojson.org/>). *geojson.org*. Retrieved August 7, 2022.
69. "JSON-LD 1.1" (<https://www.w3.org/TR/json-ld/>). *World Wide Web Consortium*. July 16, 2020. Retrieved June 17, 2022.
70. "JSON-LD - JSON for Linking Data" (<https://json-ld.org/>). *json-ld.org*. Retrieved August 7, 2022.
71. "JSON-RPC" (<https://www.jsonrpc.org/>). *jsonrpc.org*. Retrieved June 17, 2022.
72. "JsonML (JSON Markup Language)" (<http://www.jsonml.org/>). *JsonML.org*. Retrieved June 17, 2022.
73. McKamey, Stephen (June 14, 2022), *JsonML* (<https://github.com/mckamey/jsonml>), retrieved August 7, 2022
74. "FasterXML/smile-format-specification: New home for Smile format" (<https://github.com/FasterXML/smile-format-specification>). *GitHub*. Retrieved June 17, 2022.
75. Gupta, Ayush (February 10, 2019). "Understanding Smile — A data format based on JSON" (<https://medium.com/code-with-ayush/understanding-smile-a-data-format-based-on-json-29972a37d376>). *Code with Ayush*. Retrieved August 7, 2022.
76. "Universal Binary JSON Specification – The universally compatible format specification for Binary JSON" (<https://ubjson.org/>). *ubjson.org*. Retrieved June 17, 2022.
77. "UBJSON - JSON for Modern C++" (https://json.nlohmann.me/features/binary_formats/ubjson/). *json.nlohmann.me*. Retrieved August 7, 2022.
78. Bryan Feuling "Repeatability, Reliability, and Scalability Through GitOps" (https://www.google.com/books/edition/Repeatability_Reliability_and_Scalabilit/ZDggQEAAQBAJ). 2021. p. 129.
79. William Hegedus. "Mastering Prometheus" (https://www.google.com/books/edition/Mastering_Prometheus/vhr9EAAQBAJ). 2024. p. 194 - 198.
80. Eric Liang and Aaron Davidson "Declarative Infrastructure with the Jsonnet Templating Language" (<https://www.databricks.com/blog/2017/06/26/declarative-infrastructure-jsonnet-templating-language.html>). 2017.

External links

- [Official website \(<https://json.org/>\)](https://json.org/) 

Retrieved from "<https://en.wikipedia.org/w/index.php?title=JSON&oldid=1324092805>"